Swinburne University of Technology

Faculty of Engineering, Computing and Science

Assignment 2

COS30020 Web Application Development

Semester 2, 2026

Due: Week 12 | **Assignment Worth**: 50% of total marks

Overview

This assignment continues the project you defined in Assignment 1. You will keep building on the same idea, the same problem, the same catalogue/activity/community concept, but move the system from a flat text file to a real MySQL database, add an Admin account type, and use PHP sessions to properly gate access.

Make a copy of your Assignment 1 submission and continue from there. Keep the page names, folder structure and general design from Assignment 1 so the site reads as one continuous system, not two unrelated projects. You're strongly encouraged to place reusable logic (DB connection, session checks, upload handling) in separate PHP files and include() them where needed, it will make debugging much easier.

Note: Wherever the brief below refers to "the database", "your item", "your activity" or "your contribution", use the terminology you settled on for your own project in Assignment 1 (e.g. rename smart_feature_table's fields to fit your domain).

Part 1: Database Table Creation

Create a database for your project programmatically, embed the code in main.php (or an included setup file) so the database is created automatically if it doesn't already exist. Create five tables inside it, each only if it does not already exist, using CREATE TABLE IF NOT EXISTS.

1. user_table: stores user profile information

- **Email:** varchar(50), not null, primary key
- **first_name:** varchar(50), not null
- **last_name:** varchar(50), not null
- **dob:** date, null
- **gender:** varchar(6), not null
- **hometown:** varchar(50), not null
- **profile_image:** varchar(100), null, path to the stored profile photo

2. account_table: stores login credentials

- **email:** varchar(50), not null, foreign key referencing user_table.email, with ON UPDATE/ON DELETE CASCADE so changes to the user's email propagate here
- **password:** varchar(255), not null, always store a hash, never plain text
- **type:** varchar(5), not null, either "admin" or "user"

3. smart_feature_table: stores the data behind your Smart Feature page

- **id:** integer, not null, auto_increment, primary key
- **item_name:** varchar(50), not null, the primary name/title of the entry
- **item_category:** varchar(50), not null, a category or secondary label for the entry
- **item_image:** varchar(100), null, path to the stored image
- **description_file:** varchar(100), null, path to a downloadable PDF description
- **status:** varchar(10), not null, "pending", "approved" or "rejected"

4. activity_table: stores users registered for an activity/booking

- **id**: integer, not null, auto_increment, primary key
- **email:** varchar(50), not null
- **first_name:** varchar(50), not null
- **last_name:** varchar(50), not null
- **date:** date, null
- **time:** time, null
- **contact_number:** varchar(15), null
- **status:** varchar(10), not null, "pending", "approved" or "rejected"

5. community_table: stores user contributions to your Community/Showcase page

- **id:** integer, not null, auto_increment, primary key
- **first_name:** varchar(50), not null
- **last_name:** varchar(50), not null
- **contribution_image:** varchar(100), null, path to the stored photo/video
- **related_activity:** varchar(50), not null, the activity/service the contribution relates to
- **status:** varchar(10), not null, "pending", "approved" or "rejected"

Note: Write code to populate all five tables with at least 4 sets of dummy data each. Include one admin record in user_table and account_table with login admin@swin.edu.my / password admin. Feel free to add more sample data, it will help you test the pages below.

Part 2: Register and Login Functions

Task 2.1 – Registration Page *(registration.php)*

Before saving a new registration to the database:

- Check whether the email already exists in the database
- Hash the password before storing it (never store plain text)
- Save the submitted information into user_table and account_table

Task 2.2 – Login Page *(login.php)*

On form submission:

- Validate the submitted password against the stored hash
- If the credentials are correct, set session variable(s) storing the account's type and email, then redirect to main_menu.php

Part 3: User Interaction Pages

Task 3.1 – Main Menu Page *(main_menu.php)*

Amend the Assignment 1 version so the user must be logged in with type = "user" to access the page, otherwise redirect to login.php. Use the session variables set in Task 2.2 to check this.

Task 3.2 – Amend Pages to Read from the Database

Update the following existing Assignment 1 pages so their content comes from the database instead of static/text-file content, and so each requires a logged-in user:

- activities.php: activity listings sourced from activity_table
- community_detail.php: a single contribution's detail sourced from community_table
- profile.php: the logged-in user's own details sourced from user_table

Task 3.3 – Update Profile Page *(update_profile.php)*

Add to the existing page from Assignment 1:

- User must be logged in as type "user"
- Load the user's current information from the database into the form on page load
- Validate input and display clear error messages on failure
- Save changes back to user_table, and update account_table too if the email address changes

The page should also let the user:

- Upload a new profile photo (jpg/jpeg/png only, max 5MB), stored under profile_images/
- Upload a supporting document, e.g. a resume, certificate, or anything relevant to your project (PDF only, max 7MB), stored under resume/

Note: Store the file path (not the file itself) in user_table.

Task 3.4 – Smart Feature Contribution Page *(smart_feature_add.php)*

Lets a logged-in user contribute an entry that will feed your Smart Feature (Task 5.1).
Requirements:

- User must be logged in as type "user", otherwise show a page directing them to login.php
- The submitted information is saved into smart_feature_table with status = "pending"
- Upload a PDF description (PDF only, max 7MB), stored under smart_feature_description/, path saved to smart_feature_table
- On success, display a clear confirmation message to the user

Part 4: Admin Interaction Pages

Task 4.1 – Admin Main Menu *(main_menu_admin.php)*

Loaded when an admin logs in. Similar layout to the user main menu, but with a visibly different (darker) navigation bar so it's clearly distinguishable.

Req-1: User must be logged in with type = "admin", otherwise redirect to login.php.

Req-2: Darker navigation bar, a home button, a logout button, and 2 admin options.

Req-3: All links/buttons work as expected.

Task 4.2 – Manage Users' Accounts Page *(manage_accounts.php)*

Lets the admin add, edit and delete user accounts. Deleting a user must remove the record from both user_table and account_table.

Task 4.3 – Manage Community Contributions Page *(manage_community.php)*

Lets the admin manage community_table entries: add, edit, delete, and approve/reject.

Req-1: User must be logged in with type = "admin", otherwise redirect to login.php.

Req-2: Display all contributions in a table, with options to accept/reject and to delete.

Approval behaviour:

- A new entry's status is "pending" until reviewed
- If status is "pending", show both Approve and Reject buttons
- If status is "approved", disable Approve and enable Reject
- If status is "rejected", disable Reject and enable Approve

Task 4.4 – Manage Activity Registrations Page *(manage_activities.php)*

Let the admin manage activity_table entries: add, edit, delete, and approve/reject, with the same approval behaviour as Task 4.3 above.

Req-1: User must be logged in with type = "admin", otherwise redirect to login.php.

Req-2: Display all registrations in a table, with options to accept/reject and to delete.

Task 4.5 – About Page *(about.php)*

A report page answering, in bullet points:

- What tasks have you not attempted or not completed?
- Which parts did you have the most trouble with?
- What would you do better next time?
- What extension features or extra challenges did you attempt?
- Link to your video presentation

Also include a link back to the home page (index.php).

Part 5: Extension Task

Task 5.1 – Smart Feature Page *(smart_feature.php)*

Build the advanced feature you reserved a spot for in Assignment 1's main menu, an identification tool, a recommendation engine, a calculator, or similar, appropriate to your project. How it works is up to you: from a simple lookup/matching approach through to a genuine image-recognition or external API integration (the latter counts toward Innovation & Enhancements marks).

Output, on a successful match:

- The item's name and category from smart_feature_table
- Its stored image
- A downloadable PDF description

If no match is found, display a clear message such as "No matching data available" instead of an error.

Req: Data is retrieved from smart_feature_table, and only "approved" entries should be eligible to appear as a result.

Assignment Submission

Build directly on your Assignment 1 folder. Additional folders needed for Assignment 2:

- profile_images/: profile photos
- resume/: uploaded supporting documents from Task 3.3
- smart_feature_description/: PDF descriptions from Tasks 3.4 and 5.1
- *.php files stay in the project root, as in Assignment 1

Note: All links, including to any files, must be relative. Absolute links break when files are transferred for marking, if links are broken, or the assignment doesn't run when tested, the result will be a fail for this assignment.

Submit individually on Canvas. Zip all source code files. Upload your demonstration video to YouTube as unlisted (maximum 7 minutes this time, since there's more to show) and link it from about.php. You may submit more than once before the due date, the latest submission overwrites the previous one.

- Test your website before submission.
- Full marks for a task will not be awarded if there are errors, or if usability isn't considered properly.
- If your assignment cannot run, your result will be 0 marks.

Marking Scheme

Task	Criteria	Marks
Part 1: Database Table Creation	• Database and 5 tables created programmatically with CREATE TABLE IF NOT EXISTS • Correct fields, types, keys and constraints for each table • Tables pre-populated with dummy data, incl. seeded admin account	**8**
Part 2: Registration & Login	• Registration: duplicate check, password hashing, correct table writes • Login: password verified against hash, session variables set correctly	**12**
Part 3.1–3.2: Main Menu & DB-Driven Pages	• main_menu.php gated by session (type = "user") • activities.php, community_detail.php and profile.php correctly read from the database • Ungated/incorrect access is handled with a redirect to login	**13**
Part 3.3: Update Profile Page	• Existing data loaded into the form on page load • Input validation with clear error messages • Photo upload (jpg/jpeg/png, ≤5MB) and document upload (pdf, ≤7MB) both working • user_table (and account_table on email change) updated correctly	**10**
Part 3.4: Smart Feature Contribution Page	• Session-gated to logged-in users • Entry + PDF upload (≤7MB, pdf only) saved to the database • Clear success/failure message shown to the user	**7**
Part 4.1–4.2: Admin Menu & Manage Accounts	• Admin main menu correctly gated and styled distinctly from the user menu • Add/edit/delete works on user_table with correct cascade to account_table	**9**
Part 4.3: Manage Community Contributions	• Table view of all contributions with add/edit/delete • Approve/reject workflow correctly toggles button state by status	**8**
Part 4.4: Manage Activity Registrations	• Table view of all registrations with add/edit/delete • Approve/reject workflow correctly toggles button state by status	**8**
Part 4.5: About Page	• All required questions answered, including reflection on the build • Video link included; link back to home page works	**3**
Part 5: Extension: Smart Feature Output	• Feature produces a meaningful result from smart_feature_table data • Handles the "no match found" case gracefully • Downloadable PDF description included in the output	**7**

Security, Code and Innovation (15 marks)

This is where students who go further are rewarded, for secure, well-organised code, and genuine extra effort including tools or techniques not covered in lectures.

Task	Criteria	Marks
Security & Code Quality	• Passwords always hashed, never stored or compared in plain text • Queries protected against SQL injection (prepared statements/parameterised queries) • Sessions and file uploads validated server-side, not just in the browser • Sensible code organisation (e.g. shared includes for DB connection/session checks)	**8**
Innovation & Enhancements	• Extra effort/initiative beyond the brief • Ideas, features or technology not covered in lectures (e.g. AJAX-driven admin approvals, a real image-matching/ML approach for the Smart Feature, pagination, search/filter, responsive design) • Enhancements that genuinely fit the student's chosen project from Assignment 1	**7**

Total: 100 marks

* The marking scheme is a guide, good coding and testing practice is expected throughout.

* Full marks for a task will not be awarded if there are errors, or if usability (UI/UX) isn't considered properly.

* If your assignment cannot be properly tested, or cannot run, your result will be 0 marks for this assignment.